



Звіт про оригінальність

● Оцінка схожості

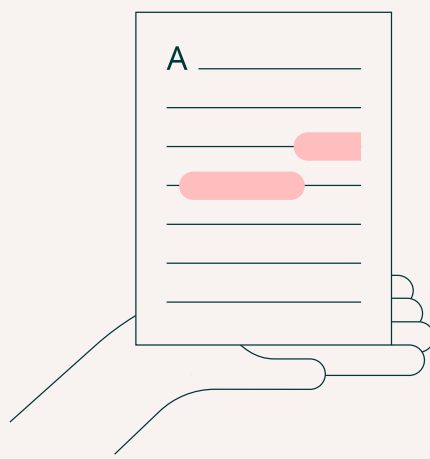
% 3

● Ризик плагіату

НИЗЬКИЙ

👤 Offiks 🕒 2026-06-21 20:45

Посилання на звіт: 1bj3p / Посилання користувача: sjWr



Ось вона – Ваша звіт про оригінальність!

Ми раді повідомити, що перевірка вашого документа завершена, і результати вже готові! Наші алгоритми старанно працювали, щоб знайти збіги в наших базах даних.

На наступних сторінках ви знайдете результати перевірки:

Бали

Збіги

Посилання

Ваш документ було перевірено за такими джерелами:

- ☒ База даних інтернет-джерел
- ☐ База даних наукових статей
- ☐ Глибока перевірка (наш вдосконалений алгоритм)

Бали



● Збіги тексту	3%
● Перефразування	0%
● Цитований текст	0%
● Неправильне цитування	0%
● Збігів не знайдено	97%

Ризик плагіату

НИЗЬКИЙ

Ризик плагіату вказує, як збіги тексту розподілені по документу. Вищий ризик виникає, коли збіги з'являються близько один до одного, наприклад, у тому самому абзаці або розділі.

Оцінка схожості

Оцінка схожості показує, скільки слів або символів у вашому документі збігаються з текстами інших документів, включаючи перефразовані тексти або неправильні цитати.



3

Збіги

3 МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ ОДЕСЬКИЙ 4 НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ І.І.МЕЧНИКОВА Факультет математики, фізики та інформаційних
технологій Кафедра математичного забезпечення комп'ютерних систем

КУРСОВИЙ ПРОЄКТ з освітнього компоненту «Програмування» на тему: СТВОРЕННЯ
ІЄРАРХІЇ КЛАСІВ НА ТЕМУ «БУФЕТ»

Студента 1 курсу, денна форма навчання спеціальність F7 «Комп'ютерна інженерія»

Янчевського Кирила Сергійовича Керівник: к.ф.- м.н., доц. Антоненко О. С.

3 Національна шкала: _____

3 Кількість балів: _____

ECTS: _____

Дата захисту: _____

Члени комісії ____ Антоненко О.С.

2 (підпис) (прізвище та ініціали)

2 ____ 2 Шестопалов С.В.

2 (підпис) (прізвище та ініціали)

2 ____ 2 _____

2 (підпис) (прізвище та 4 ініціали)

4 ОДЕСА – 2026

ЗМІСТ

стор.

ВСТУПЗ

ЗАВДАННЯ4

1.ТЕОРЕТИЧНА ЧАСТИНА5

1.1Поняття ООП5

1.2Опис об'єктної частини6

2.ПРАКТИЧНА ЧАСТИНА7

2.1 Опис класів7

2.2 Інтерфейс11

ВИСНОВКИ19

СПИСОК ЛІТЕРАТУРИ20

Додаток А. Діаграма класів21

Додаток Б. Код основних класів22

ВСТУП

Метою цієї роботи є розробка ієрархії класів для моделювання роботи «Буфету» та її **5 застосування для вирішення конкретної прикладної задачі згідно з варіантом.** У процесі проектування системи важливо практично реалізувати ключові концепції об'єктно-орієнтованого програмування: інкапсуляцію, успадкування та поліморфізм. Кінцевий програмний продукт має представляти собою систему взаємопов'язаних класів, які детально описують **1 внутрішню структуру та бізнес-процеси буфету.**

Для коректної роботи з об'єктами необхідно передбачити відповідні конструктори, поля та методи. Доступ до полів класу має бути закритим, а взаємодія з ними повинна відбуватися виключно через публічні методи. Структура програми має базуватися на зв'язках успадкування та композиції. Окрім цього, для забезпечення динамічного поліморфізму слід інтегрувати механізм віртуальних методів, а також задіяти абстрактні класи чи чисті віртуальні функції там, де це необхідно.

ЗАВДАННЯ

Варіант 236. Створення ієрархії класів на тему «Буфет»

У роботі необхідно реалізувати класи, що описують структуру буфету, меню страв,

відвідувачів та продавців. Потрібно передбачити можливість роботи з різними категоріями страв, збереження інформації про їхню назву, ціну, доступну кількість. Також необхідно реалізувати взаємодію між класами, перевірку коректності введених даних та виконання основних операцій, пов'язаних із функціонуванням буфету: процеси замовлення, оплати та контролю.

ТЕОРЕТИЧНА ЧАСТИНА

Поняття ООП

Об'єктно-орієнтований **1** програмування (ООП) – це парадигма розробки, де програма будується як мережа взаємодіючих об'єктів [1]. **1** Будь-який об'єкт є екземпляром певного класу, а самі класи організовані в ієрархічну структуру. Спочатку розробник описує класи, на базі яких під час виконання коду генеруються конкретні об'єкти. Завдяки можливості розширювати базові класи новими, вибудовується розгалужена класова ієрархія [2].

Фундаментом цього стилю програмування є об'єктна модель, яка тримається на трьох головних концепціях [2]:

інкапсуляція;

наслідування;

поліморфізм.

Інкапсуляція: Цей механізм полягає в об'єднанні інформації (даних) та функцій (методів), що з нею працюють, у межах одного об'єкта. Це забезпечує надійний захист внутрішньої структури від небажаного зовнішнього втручання або помилкового використання [1,2].

Наслідування: Цей процес надає можливість одному об'єкту переймати властивості та поведінку іншого. Таким чином, об'єкт може не лише копіювати існуючі характеристики, але й доповнювати їх своїми унікальними рисами [1,2].

Поліморфізм: Це властивість, що дозволяє застосовувати одне й те саме ім'я для опису спільного класу дій, вирішуючи при цьому різні технічні завдання. **6** На практиці це означає, що об'єкт здатний самостійно обирати потрібний внутрішній метод обробки інформації, орієнтуючись на тип даних, які він отримує у повідомленні [1,2].

Опис об'єктної частини

Програма пропонує систему розподілу прав для трьох категорій користувачів: відвідувач, продавець та інспектор. Можливість виконання тих чи інших дій всередині

програми безпосередньо залежить від вибору відвідувача та продавця користувачем, перевірки його балансу (для здійснення замовлення їжі чи напоїв).

Відвідувач взаємодіє з інтерфейсом буфету для перегляду актуального меню (асортименту гарячих страв, холодних страв та напоїв) та здійснення покупки. Він купує продукцію, якщо його баланс є достатнім для оплати обраних позицій, чи, якщо баланс є недостатнім, має вибір скоригувати замовлення або зупинити вибір страв.

Головним елементом системи є меню буфету, що складається з базового абстрактного класу страв та трьох спеціалізованих класів-нащадків: гарячі страви, холодні страви та напої. Кожна позиція меню характеризується набором закритих параметрів (назва, вартість, поточна кількість товару). Робота із асортиментом здійснюється безпосередньо через інтерфейс програми, який дозволяє переглядати доступні товари та перевіряти наявність необхідної кількості одиниць.

ПРАКТИЧНА ЧАСТИНА

2.1 Опис класів

Діаграму класів наведено в додатку А. Розглянемо структуру кожного класу.

Абстрактний клас «Person» – людина:

Захищені поля: fullName, position.

Конструктори:

Конструктор без параметрів;

Конструктор з параметрами.

Методи:

Чисто віртуальна функція printInfo.

Клас «Visitor», нащадок класу «Person» – відвідувач:

Приватні поля: money.

Конструктори:

Конструктор без параметрів;

Конструктор з параметрами.

Методи:

getMoney – повертає кількість грошей об'єкта;

changeMoney – змінює кількість грошей об'єкта;

Перевизначення printInfo – виводить інформацію про об'єкт.

Клас «Seller», нащадок класу «Person» – продавець:

Приватні поля: badProbability, stolen, cheated, arrested.

Конструктори:

Конструктор без параметрів;

Конструктор з параметрами.

Методи:

getProb – повертає шанс шахрайства;

stole – повертає скільки вкрадено;

isArrested – повертає чи арестован об'єкт;

testCheat – повертає чи зшахрував об'єкт;

cheat – змінює поле cheated на true;

getArrested – змінює поле arrested на true;

steal – змінює кількість вкрадених грошей;

printName – виводить ім'я об'єкта;

Перевизначення printInfo – виводить інформацію про об'єкт.

Клас «Inspector», нащадок класу «Person» – інспектор:

Приватні поля: organization.

Конструктори:

Конструктор без параметрів;

Конструктор з параметрами.

Методи:

Перевизначення `printInfo` – виводить інформацію про об'єкт.

Абстрактний клас «Dish» – страва:

Приватні поля: `name`, `price`, `quantity`, `tempSold`.

Конструктори:

Конструктор без параметрів;

Конструктор з параметрами.

Методи:

`getName` – повертає назву об'єкта;

`getPrice` – повертає вартість об'єкта;

`getQuantity` – повертає кількість порцій об'єкта;

`getTempSold` – повертає поле `tempSold`;

`increaseQuantity` – збільшує кількість порцій об'єкта;

`increaseTempSold` – збільшує значення поля `tempSold`;

`resetTempSold` – змінює значення поля `tempSold` на 0;

`printInfo` – виводить інформацію про об'єкт;

Чисто віртуальна функція `getType`.

Клас «HotDish», нащадок класу «Dish» – гаряча страва:

Конструктори:

Конструктор з параметрами.

Методи:

Перевизначення `getType` – повертає вид страви.

Клас «ColdDish», нащадок класу «Dish» – холодна страва:

Конструктори:

Конструктор з параметрами.

Методи:

Перевизначення `getType` – повертає вид страви.

Клас «Beverage», нащадок класу «Dish» – напій:

Конструктори:

Конструктор з параметрами.

Методи:

Перевизначення `getType` – повертає вид страви.

Клас «Memory» – клас для безпечного зберігання об'єктів:

Статичні приватні поля: `visitors`, `sellers`, `dishes`, `selectedVisitor`, `selectedSeller`.

Методи:

`addVisitor` – додає відвідувача;

`addSeller` – додає продавця;

`addDish` – додає страву;

`deleteVisitor` – видаляє відвідувача;

`deleteSeller` – видаляє продавця;

`deleteDish` – видаляє страву;

`getVisitor` – повертає відвідувача за індексом;

`getSeller` – повертає продавця за індексом;

`getDish` – повертає страву за індексом;

`getVisitorCount` – повертає кількість відвідувачів;

`getSellerCount` – повертає кількість покупців;

`getDishCount` – повертає кількість об'єктів страв;

`selectVisitor` – змінює вибраного відвідувача за індексом;

selectSeller – змінює вибраного продавця за індексом;

getSelectedVisitor – повертає вибраного відвідувача;

getSelectedSeller – повертає вибраного продавця;

nullSeller – прибирає вибір продавця;

returnDishes – повертає всі страви;

returnSellers – повертає всіх продавців.

2.2 Інтерфейс

Готовий проєкт розміщено в [4]. Під час запуску програми користувач бачить початковий інтерфейс (рис. 2.1).

Рисунок 2.1 – Початковий інтерфейс

Користувач потрапляє сюди коли запускає програму, виходить з вибору дії або завершає цю дію. Після того як користувач вибере дію яку хоче зробити, йому буде виведено інформацію стосовно неї. Якщо користувач зразу вибере 3 варіант, попередньо не вибрав покупця чи продавця, буде виведено помилку і початковий інтерфейс (рис. 2.2, 2.3).

Рисунок 2.2 – Помилка коли не вибраний покупець

Рисунок 2.3 – Помилка коли не вибраний продавець

Якщо користувач вибере 1 варіант, буде виведено усіх покупців для вибору (рис. 2.4) і опція повернутися до початкового інтерфейсу.

Рисунок 2.4 – Вибір покупця

При виборі одного з покупців користувача поверне до початкового інтерфейсу, але тепер біля опції вибору покупців буде відображено нинішній вибір (рис. 2.5).

Рисунок 2.5 – Початковий інтерфейс з вибраним покупцем

Вибір продавця проходить так само (рис. 2.6, 2.7).

Рисунок 2.6 – Вибір продавця

Рисунок 2.7 – Початковий інтерфейс з вибраним продавцем

Тепер у випадку вибору 3 користувача зустрине меню (рис. 2.8) з опціями вибору страви

та опцією виходу до початкового інтерфейсу.

Рисунок 2.8 – Меню

При виборі страви також зазначається кількість порцій (рис. 2.9).

Рисунок 2.9 – Вибір кількості порцій

Якщо вибрати більше порцій чим є, буде виведена помилка (рис. 2.10).

Рисунок 2.10 – Помилка коли недостатньо порцій

Після коректного вибору страви та кількості порцій користувач зможе продовжувати замовлення чи завершити його (рис. 2.11). Також кількість страв буде тимчасово зменшена. А вартість замовлення збільшиться.

Рисунок 2.11 – Меню з опцією завершення замовлення

Якщо зробити замовлення коли у відвідувача недостатньо коштів, буде виведено помилку (рис. 2.12) з можливістю скоригувати замовлення чи повністю закрити меню з поверненням до початкового інтерфейсу.

Рисунок 2.12 – Помилка коли недостатньо коштів

Якщо коштів вистачило, то замовлення завершиться (рис. 2.13): кількість порцій і кількість коштів у користувача зменшиться, кількість коштів у касі збільшиться з шансом (рис. 2.14) того, що продавець привласнить собі частину коштів.

Рисунок 2.13 – Завершення замовлення

Рисунок 2.14 – Шанс недоброчесності продавця

Якщо в початковому інтерфейсі вибрати варіант 4, то день завершиться і прийде інспектор (рис. 2.15).

Рисунок 2.15 – Завершення дня

Інспектор перевіряє кількість проданих страв та кількість грошей в касі. Якщо порушень не знайдено (рис. 2.16), то день завершується звичайним чином і кількість страв поповнюється (рис. 2.17).

Рисунок 2.16 – Перевірка інспектора без вияву порушень

Рисунок 2.17 – Новий день

Якщо ж інспектор виявляє порушення, починається перевірка кожного продавця з послідувачим арештом (рис. 2.18) і неможливість продажу страв заарештованим (рис. 2.19).

Рисунок 2.18 – Перевірка інспектора з виявом порушень

Рисунок 2.19 – Спроба вибрати заарештованого продавця

Також в користувача є опції створення своїх страв (рис. 2.20) та додання людей (рис. 2.21) чи видалення (рис. 2.22).

Рисунок 2.20 – Створення страви

Рисунок 2.21 – Додання людини

Рисунок 2.22 – Видалення людини

Також під час будь-якого вводу відбувається дві перевірки: чи є введений символ числом (рис. 2.23) та чи можливий це варіант(рис. 2.24).

Рисунок 2.23 – Перевірка на число

Рисунок 2.24 – Перевірка на можливість варіанту дії

ВИСНОВКИ

Результатом курсової роботи стало створення консольної програми «Буфет» мовою C++ із застосуванням об'єктно-орієнтованого програмування. Архітектура проекту містить ієрархічну систему класів, що описують страви, меню, відвідувачів і продавців, а також забезпечують їхню коректну взаємодію.

Під час проектування було практично застосовано інкапсуляцію, поліморфізм та наслідування. Кожен компонент програми містить набір конструкторів, полів і методів. Серед головних функцій реалізовано: консольне керування, створення об'єктів людей і страв, перевірку коректності введення інформації, а також функцію інспекторського контролю за порушеннями.

Цей проєкт дозволив покращити практичні вміння роботи з мовою C++, базовими конструкціями класів, методами та консоллю. Написана програма підтверджує, що засоби ООП є чудовим інструментом для побудови повноцінних систем керування об'єктами громадського харчування.

СПИСОК ЛІТЕРАТУРИ

Wikipedia. Об'єктно-орієнтоване **7** програмування [Електронний ресурс] – Режим

доступу: https://uk.wikipedia.org/wiki/Об'єктно-орієнтоване_програмування

Web Library. Основні поняття ООП. [Електронне видання] – Режим доступу:
<http://weblib.com.ua/blog/osnovni-ponyattya-oop>

Репозиторій GitHub на курсовий – Режим доступу:
<https://github.com/Yanchik228/CourseProject>

Додаток А. Діаграма класів

Рисунок А.1 – Діаграма класів

Додаток Б. Код основних класів

```
#include "Person.h"

#include <stdexcept>

#include <iostream>

Person::Person()

: fullName("Невідомо") {}

Person::Person(const std::string& name)

: fullName(name)

{

if (fullName.empty()) {

throw std::invalid_argument("ПІБ не може бути порожнім!");

}

}
```

Лістинг Б.1 – Клас Person

```
#include "Visitor.h"

#include <iostream>

Visitor::Visitor()

: Person("Невідомо"), money(0) {}
```

```

Visitor::Visitor(const std::string& name, int money)
: Person(name), money(money)
{
    if (money < 0) {
        throw std::invalid_argument("Грошей не може бути від'ємна кількість!");
    }
}

int Visitor::getMoney() const {
    return money;
}

void Visitor::changeMoney(int n) {
    money += n;
}

void Visitor::printInfo() const {
    std::cout << fullName << ", " << "грошей: " << money << " грн";
}

```

Лістинг Б.2 – Клас Visitor

```

#include "Seller.h"

#include <stdexcept>

#include <iostream>

Seller::Seller()
: Person("Невідомо"), badProbability(0.0), arrested(false), cheated(false), stolen(0) {}

Seller::Seller(const std::string& name, double prob)
: Person(name), badProbability(prob), arrested(false), cheated(false), stolen(0)

```

```

{

if (badProbability > 1.0 || badProbability < 0.0) {

throw std::invalid_argument("Неможливий шанс!");

}

}

double Seller::getProb() const {

return badProbability;

}

int Seller::getStolen() const {

return stolen;

}

bool Seller::isArrested() const {

return arrested;

}

bool Seller::testCheat() const {

return cheated;

}

void Seller::cheat() {

cheated = true;

}

void Seller::steal(int n) {

stolen += n;

}

void Seller::getArrested() {

```



```

arrested = true;

}

void Seller::printName() const {

std::cout << fullName;

}

void Seller::printInfo() const {

std::cout << fullName << ", " << "шанс порушення: " << badProbability * 100 << "%";

if (arrested) {

std::cout << " [Арештован]";

}

}

```

Лістинг Б.3 – Клас Seller

```

#include "Inspector.h"

#include <stdexcept>

#include <iostream>

Inspector::Inspector()

: Person("Невідомо"), organization("Немає") {}

Inspector::Inspector(const std::string& name, const std::string& org)

: Person(name), organization(org)

{

if (organization.empty()) {

throw std::invalid_argument("Поле організації не може бути порожнім!");

}

}

```

```

void Inspector::printInfo() const {

std::cout << fullName << " (" << organization << ")";

}

```

Лістинг Б.4 – Клас Inspector

```

#include "Dish.h"

#include <stdexcept>

#include <iostream>

#include <cmath>

Dish::Dish()

: name("Невідомо"), price(0), quantity(0), tempSold(0) {}

Dish::Dish(const std::string& name, int price, int quantity)

: name(name), price(price), quantity(quantity), tempSold(0)

{

if (name.empty()) {

throw std::invalid_argument("Назва не може бути порожнім!");

}

if (price < 0) {

throw std::invalid_argument("Вартість не може бути негативною!");

}

if (quantity < 0) {

throw std::invalid_argument("Кількість не може бути негативною!");

}

}

std::string Dish::getName() const {

```

```

return name;

}

int Dish::getPrice() const {

return price;

}

int Dish::getQuantity() const {

return quantity;

}

int Dish::getTempSold() const {

return tempSold;

}

```

Лістинг Б.5 – Клас Dish, аркуш 1

```

void Dish::increaseQuantity(int increase) {

if (increase <= 0) return;

else {

quantity += increase;

}

}

void Dish::increaseTempSold(int increase) {

if (increase <= 0) return;

else {

tempSold += increase;

quantity -= increase;

}

}

```

```

}

void Dish::resetTempSold() {

tempSold = 0;

}

void Dish::printInfo() const {

std::cout << "[" << getType() << "]" " << name << ", ціна: " << price << " грн, кількість: " <<
quantity << " порцій";

}

```

Лістинг Б.5 – Клас Dish, аркуш 2

```

HotDish::HotDish(const std::string& name, int price, int quantity)

: Dish(name, price, quantity) {}

std::string HotDish::getType() const {

return "Гаряча страва";

}

```

Лістинг Б.6 – Клас HotDish

```

ColdDish::ColdDish(const std::string& name, int price, int quantity)

: Dish(name, price, quantity) {}

std::string ColdDish::getType() const {

return "Холодна страва";

}

```

Лістинг Б.7 – Клас ColdDish

```

Beverage::Beverage(const std::string& name, int price, int quantity)

: Dish(name, price, quantity) {}

std::string Beverage::getType() const {

```

```
return "Напій";
```

```
}
```

Лістинг Б.8 – Клас Beverage

```
#include "Memory.h"
```

```
std::vector<std::shared_ptr<Visitor>> Memory::visitors;
```

```
std::vector<std::shared_ptr<Seller>> Memory::sellers;
```

```
std::vector<std::shared_ptr<Dish>> Memory::dishes;
```

```
std::shared_ptr<Visitor> Memory::selectedVisitor = nullptr;
```

```
std::shared_ptr<Seller> Memory::selectedSeller = nullptr;
```

```
void Memory::addVisitor(const std::shared_ptr<Visitor>& visitor) {
```

```
    if (visitor != nullptr) {
```

```
        visitors.push_back(visitor);
```

```
    }
```

```
}
```

```
void Memory::addSeller(const std::shared_ptr<Seller>& seller) {
```

```
    if (seller != nullptr) {
```

```
        sellers.push_back(seller);
```

```
    }
```

```
}
```

```
void Memory::addDish(const std::shared_ptr<Dish>& dish) {
```

```
    if (dish != nullptr) {
```

```
        dishes.push_back(dish);
```

```
    }
```

```
}
```

```

bool Memory::deleteVisitor(size_t index) {

if (index < visitors.size()) {

visitors.erase(visitors.begin() + index);

return true;

}

return false;

}

bool Memory::deleteSeller(size_t index) {

if (index < sellers.size()) {

sellers.erase(sellers.begin() + index);

return true;

}

return false;

}

bool Memory::deleteDish(size_t index) {

if (index < dishes.size()) {

dishes.erase(dishes.begin() + index);

return true;

}

return false;

}

std::shared_ptr<Visitor> Memory::getVisitor(size_t index) {

if (index < visitors.size()) {

return visitors[index];

```

```

}

return nullptr;

}

std::shared_ptr<Seller> Memory::getSeller(size_t index) {

if (index < sellers.size()) {

return sellers[index];

}

return nullptr;

}

```

Лістинг Б.9 – Клас Memory, аркуш 1

```

std::shared_ptr<Dish> Memory::getDish(size_t index) {

if (index < dishes.size()) {

return dishes[index];

}

return nullptr;

}

size_t Memory::getVisitorCount() {

return visitors.size();

}

size_t Memory::getSellerCount() {

return sellers.size();

}

size_t Memory::getDishCount() {

```

```

return dishes.size();

}

void Memory::selectVisitor(size_t index) {

if (index < visitors.size()) {

selectedVisitor = visitors[index];

}

}

void Memory::selectSeller(size_t index) {

if (index < sellers.size()) {

selectedSeller = sellers[index];

}

}

void Memory::nullSeller() {

selectedSeller = nullptr;

}

std::shared_ptr<Visitor> Memory::getSelectedVisitor() {

return selectedVisitor;

}

std::shared_ptr<Seller> Memory::getSelectedSeller() {

return selectedSeller;

}

std::vector<std::shared_ptr<Dish>> Memory::returnDishes() {

return dishes;

}

```



```
std::vector<std::shared_ptr<Seller>> Memory::returnSellers() {  
  
    return sellers;  
  
}
```

Лістинг Б.9 – Клас Memory, аркуш 2

Посилання

Це джерела виділених збігів у вашому документі. Кожен збіг позначено темно-зеленим числом, яке відповідає вказаному тут джерелу. Джерела впорядковані за схожістю — чим вищий бал, тим сильніше збіг.

#	Джерело	%
1	files.znu.edu.ua	0.7%
2	dspace.onu.edu.ua	0.5%
3	essuir.sumdu.edu.ua	0.4%
4	dspace.onu.edu.ua	0.3%
5	enpuir.udu.edu.ua	0.3%
6	iir.edu.ua	0.2%
7	card-file.onaft.edu.ua	0.2%



Дякуємо, що перевірили
свій документ за допомогою
Plag!